# XPLOG++: Dynamic Observability for Distributed Edges

Aman Sharma

Professor Sandip Chakraborty
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

April 30, 2025

# Outline


1. Introduction & Motivation
2. Contribution 1: Dynamic Logging
3. Contribution 2: Log Multiplexing
4. Contribution 3: Proposed CoD Architecture
5. Performance Evaluation
6. Conclusion & Future Work

# The Challenge: Observability in Distributed Systems

- Distributed systems (Microservices, Edge Computing) are complex.
- Monitoring, debugging, and performance analysis are critical.
- Traditional logging often lacks **causal consistency** across nodes.
- High logging overhead can impact performance, especially at the edge.

# The Challenge: Observability in Distributed Systems

- Distributed systems (Microservices, Edge Computing) are complex.
- Monitoring, debugging, and performance analysis are critical.
- Traditional logging often lacks **causal consistency** across nodes.
- High logging overhead can impact performance, especially at the edge.
- **Context:** 6G & Compute-on-Demand (CoD)
  - Ultra-low latency requirements.
  - Resource-constrained edge nodes (e.g., cell towers).
  - Need for efficient, insightful observability tools.

# The Challenge: Observability in Distributed Systems

- Distributed systems (Microservices, Edge Computing) are complex.
- Monitoring, debugging, and performance analysis are critical.
- Traditional logging often lacks **causal consistency** across nodes.
- High logging overhead can impact performance, especially at the edge.
- **Context:** 6G & Compute-on-Demand (CoD)
  - Ultra-low latency requirements.
  - Resource-constrained edge nodes (e.g., cell towers).
  - Need for efficient, insightful observability tools.
- **Goal:** Enhance XPLOG for 6G CoD environments focusing on efficiency and practical deployment.

# Background: XPLOG Core Concepts

**XPLOG Framework:**

- eBPF-based logging.
- Captures syscalls with low overhead.
- Aims for causally consistent logs across hosts.
- Uses Vector Clocks for inter-host ordering (at Collector).

**eBPF (Extended Berkeley Packet Filter):**

- Safe, programmable kernel monitoring.
- Attach probes to tracepoints (e.g., syscalls).

**Compute-on-Demand (CoD):**

- Utilize compute resources in network infrastructure (towers).
- Hyperlocal processing.
- Key Benefits:
  - Ultra-low latency.
  - Efficient resource use.
  - Bandwidth savings.

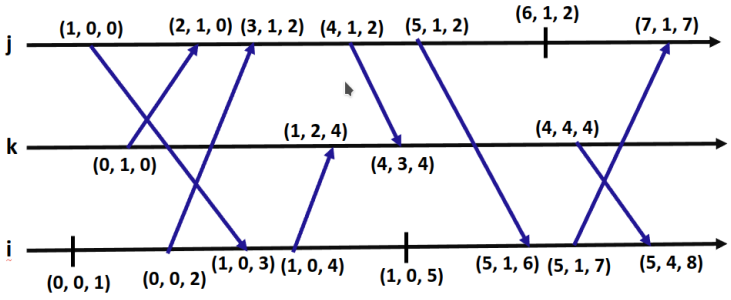



Figure: Example: Vector Clocks (Lamport & Mattern)

# Problem: Baseline Logging Overhead

- Original XPLOG logs *all* captured syscalls.
- Frequent syscalls (e.g., ‘read‘, ‘write‘) create high volume.
- Performance impact can be significant, especially for I/O heavy apps.

# Problem: Baseline Logging Overhead

- Original XPLOG logs *all* captured syscalls.
- Frequent syscalls (e.g., ‘read‘, ‘write‘) create high volume.
- Performance impact can be significant, especially for I/O heavy apps.

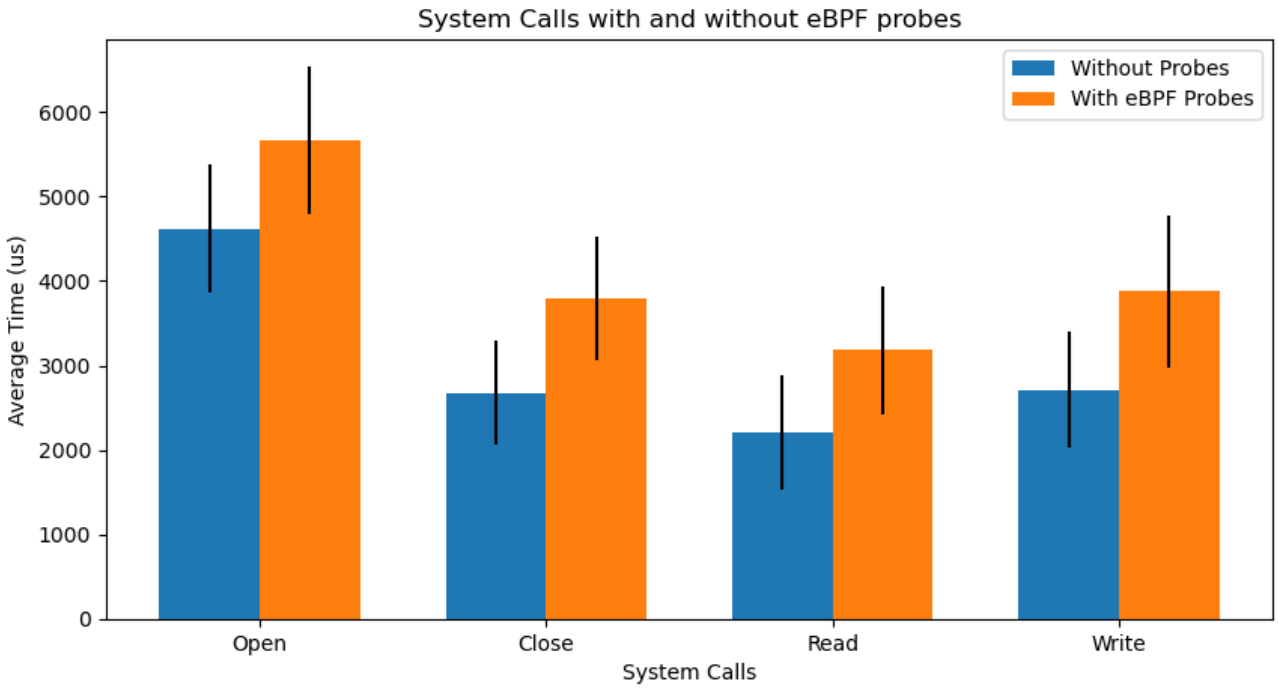



Figure: Syscall Execution Time: With vs. Without eBPF Probes (Illustrative, based on thesis Fig 3.2)

- **Need:** Selectively log only necessary syscalls.

# Solution: Dynamic Logging Mechanism (Concept)

**Core Idea:**

- User specifies which syscalls to log **at runtime**.
- Control via simple interface (named pipe/FIFO).
- eBPF probes check a shared map before deciding to log.

# Solution: Dynamic Logging Mechanism (Concept)

**Core Idea:**

- User specifies which syscalls to log **at runtime**.
- Control via simple interface (named pipe/FIFO).
- eBPF probes check a shared map before deciding to log.

**Benefits:**

- Drastically reduces log volume.
- Lowers CPU overhead (Agent & Collector).
- Decreases network bandwidth usage.
- Reduces storage requirements.
- Improves log latency.

# Solution: Dynamic Logging Mechanism (Concept)

**Core Idea:**

- User specifies which syscalls to log **at runtime**.
- Control via simple interface (named pipe/FIFO).
- eBPF probes check a shared map before deciding to log.

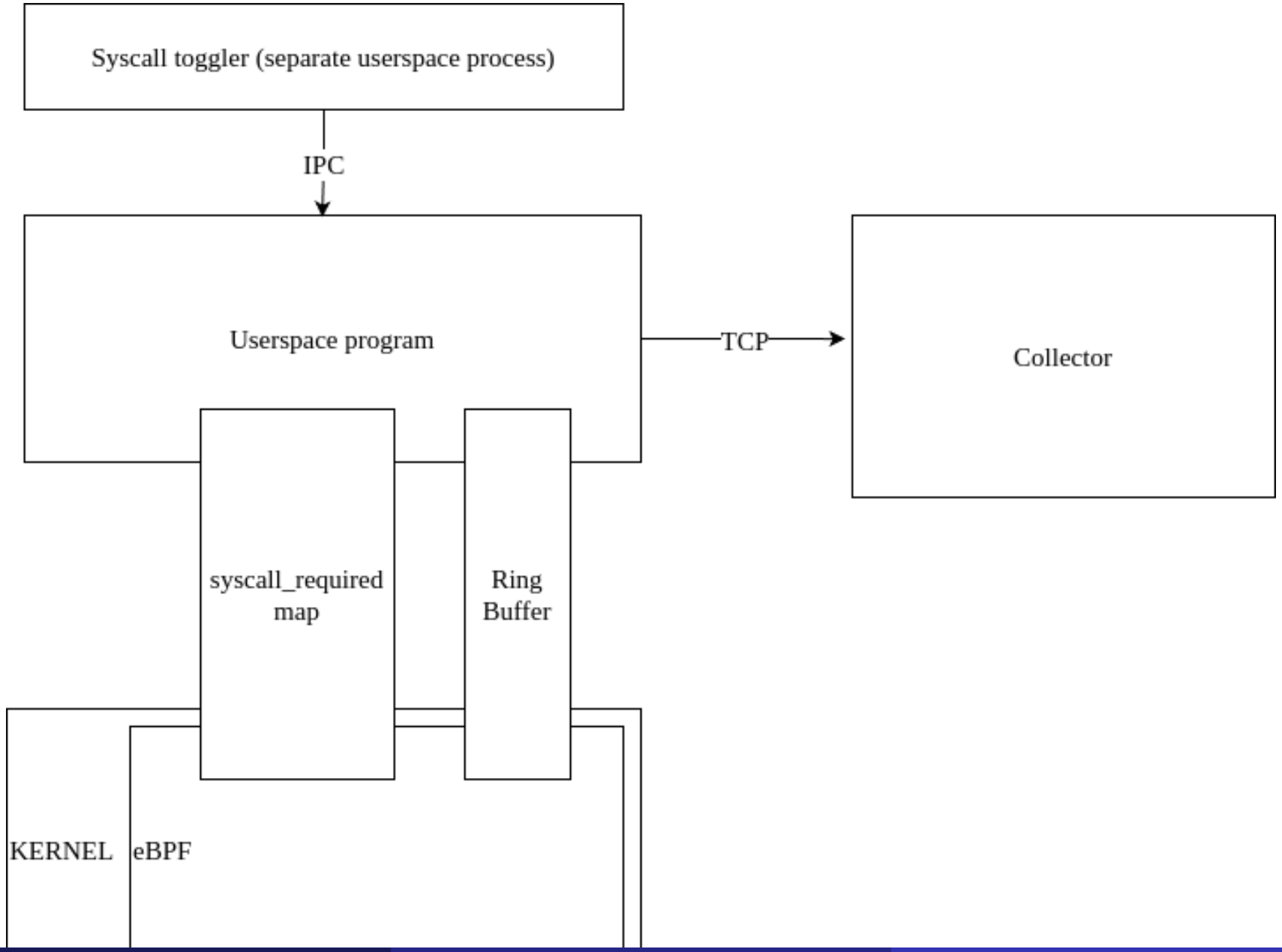
**Benefits:**

- Drastically reduces log volume.
- Lowers CPU overhead (Agent & Collector).
- Decreases network bandwidth usage.
- Reduces storage requirements.
- Improves log latency.

**Implementation Snippet:**

- User tool writes ‘syscall_num enable/disable‘ to FIFO.
- Agent (user space) listens to FIFO, updates ‘syscall_required‘ eBPF hash map.
- Syscall exit probe (kernel space) checks map: Log only if value is ’enabled’.

# Solution: Dynamic Logging Mechanism (Design)



# Problem: Diverse Services on Shared Edge Nodes

- Edge nodes often host multiple **distinct** services simultaneously:
  - Core Application (e.g., deployed via CoD)
  - System Monitoring Agents (CPU, Memory, Network metrics)
  - Security Tools
  - Platform Services
- Each service generates logs or data streams.

# Problem: Diverse Services on Shared Edge Nodes

- Edge nodes often host multiple **distinct** services simultaneously:
  - Core Application (e.g., deployed via CoD)
  - System Monitoring Agents (CPU, Memory, Network metrics)
  - Security Tools
  - Platform Services
- Each service generates logs or data streams.
- How can **one XPLOG++ Agent** efficiently collect and forward these varied streams while keeping them separate for downstream processing?

# Problem: Diverse Services on Shared Edge Nodes

- Edge nodes often host multiple **distinct** services simultaneously:
  - Core Application (e.g., deployed via CoD)
  - System Monitoring Agents (CPU, Memory, Network metrics)
  - Security Tools
  - Platform Services
- Each service generates logs or data streams.
- How can **one XPLOG++ Agent** efficiently collect and forward these varied streams while keeping them separate for downstream processing?

**Challenges without Multiplexing:**

- Mixed log streams $\rightarrow$ Impossible to analyze specific service behavior.
- Multiple Agent instances $\rightarrow$ High resource waste on constrained edge nodes.
- Multiple Agent-Collector connections $\rightarrow$ Network overhead, management complexity.

# Solution: Log Multiplexing Mechanism

**Core Idea:** Use a single Agent-Collector connection, but tag data streams by source service.

**Chosen Approach: Prepended ServiceID Byte**

- Agent assigns a unique 1-byte 'ServiceID' to each monitored service (e.g., App=0, Metrics=1, Security=2).
- Before sending data (log, metric), Agent prepends the corresponding ID: `[ID] [Original Service Data]`
- Collector reads the first byte to identify the source service.
- Routes the remaining data (original payload) to the appropriate service-specific handler.

# Solution: Log Multiplexing Mechanism

**Core Idea:** Use a single Agent-Collector connection, but tag data streams by source service.

**Chosen Approach: Prepended ServiceID Byte**

- Agent assigns a unique 1-byte 'ServiceID' to each monitored service (e.g., App=0, Metrics=1, Security=2).
- Before sending data (log, metric), Agent prepends the corresponding ID: `[ID] [Original Service Data]`
- Collector reads the first byte to identify the source service.
- Routes the remaining data (original payload) to the appropriate service-specific handler.

**Key Advantages for Diverse Edge Services:**

✓ Efficient use of single Agent instance and network connection.
✓ Minimal parsing overhead at Collector & simple implementation.
✓ Very low network overhead (1 extra byte per message).
✓ Handles varied data formats (preserves original log/metric format).

# Solution: Log Multiplexing Mechanism

**Core Idea:** Use a single Agent-Collector connection, but tag data streams by source service.

**Chosen Approach: Prepended ServiceID Byte**

- Agent assigns a unique 1-byte 'ServiceID' to each monitored service (e.g., App=0, Metrics=1, Security=2).
- Before sending data (log, metric), Agent prepends the corresponding ID: `[ID] [Original Service Data]`
- Collector reads the first byte to identify the source service.
- Routes the remaining data (original payload) to the appropriate service-specific handler.

**Key Advantages for Diverse Edge Services:**

✓ Efficient use of single Agent instance and network connection.
✓ Minimal parsing overhead at Collector & simple implementation.
✓ Very low network overhead (1 extra byte per message).
✓ Handles varied data formats (preserves original log/metric format).

**Collector Refactoring:** Object-oriented design with Handler Registry

# Log Multiplexing: Format Alternatives Comparison

Table: Comparison of Log Format Alternatives for Multiplexing (based on thesis Table 3.1)

| Criterion | JSON Encapsulation | Fixed Binary Header | Prepended Byte |
|---|---|---|---|
| Parsing Overhead (Demux) | High | Very Low | Low |
| Network Overhead | High | Low-Medium | **Very Low** |
| Implementation Complexity | Low-Medium | High | **Low** |
| Payload Format Flexibility | Medium | Low | **High** |
| Human Readability (Header) | High | Low | Low |
| Service Limit | Unlimited | Defined by Field | 256 / 65536 |

**Rationale:** Prepended Byte offers the best balance of low overhead, simplicity, and flexibility for handling diverse service streams on edge nodes.

# Vision: Practical 6G Compute-on-Demand

- Goal: Orchestrate applications on distributed edge nodes (towers) effectively.
- Requirements:
  - Standardized orchestration (e.g., Kubernetes).
  - Robust monitoring & alerting (metrics-based).
  - Deep diagnostic capabilities (log-based).
  - Standard APIs for developers (e.g., CAMARA).
  - Efficient scheduling & autoscaling for SLA adherence.

# Proposed CoD Architecture Overview

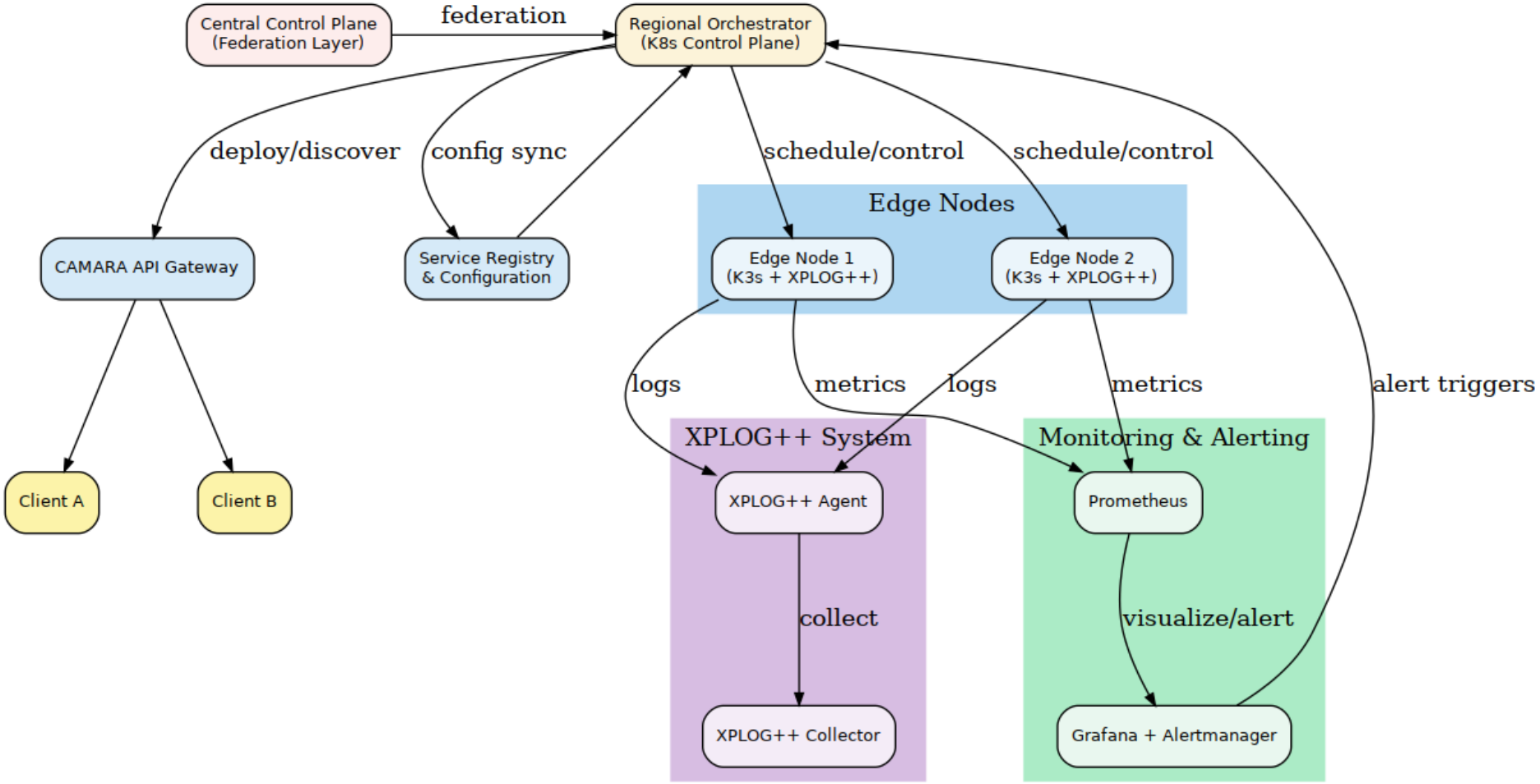



Figure: Proposed 6G CoD Architecture (based on thesis Fig 4.1)

# Key Components & XPLOG++ Role

- **Orchestration:** Kubernetes (K3s/KubeEdge) - Hierarchical (Regional + Central).
  - Leverages standard scheduling (filtering/scoring) & autoscaling (HPA/VPA).
  - *Rationale:* Maturity, predictability, ecosystem vs. RL/Swarm.
- **Standard Monitoring:** Prometheus (Metrics) + Grafana (Dashboards/Alerting).
  - Drives automated scaling and basic alerting based on SLAs.
- **APIs:** CAMARA-compliant Gateway (QoD, Edge App Mgmt, Discovery).
- **XPLOG++ Role (Agent + Collector):**
  - **Complementary** to Prometheus/Grafana.
  - Provides **deep, causally consistent logs** (via syscalls).
  - **Primary Use Cases:**
    - Root cause analysis (complex failures / performance issues).
    - Debugging distributed interactions.
    - Security auditing.
    - Generate specific metrics (if needed).
  - *Not* the primary driver for real-time autoscaling loops.

# Evaluation Goal & Setup

**Goal:** Quantify the performance impact of Dynamic Logging and Log Multiplexing.

**Setup:**

- Hardware: AMD Ryzen 5 4600H, 16GB RAM, 1Gbps LAN.
- Software: Ubuntu 22.04, Kernel 5.15, Docker.
- Workload: I/O intensive C++ app (millions of ‘open/close/read/write‘).
- Duration: 5 minutes per run.
- Metrics: CPU (%), Memory (KiB), Log Size (Bytes), Log Latency (ms).

# Evaluation Goal & Setup

**Goal:** Quantify the performance impact of Dynamic Logging and Log Multiplexing.

**Setup:**

- Hardware: AMD Ryzen 5 4600H, 16GB RAM, 1Gbps LAN.
- Software: Ubuntu 22.04, Kernel 5.15, Docker.
- Workload: I/O intensive C++ app (millions of ‘open/close/read/write‘).
- Duration: 5 minutes per run.
- Metrics: CPU (%), Memory (KiB), Log Size (Bytes), Log Latency (ms).

**Configurations Tested:**

1. **Config I:** Baseline XPLOG++ (Log all syscalls)
2. **Config II:** XPLOG++ w/ Dynamic Logging (Log only ‘open‘/‘close‘)
3. **Config III:** XPLOG++ w/ Dynamic Logging + Multiplexing (Log ‘open‘/‘close‘, ServiceID=0)

# Performance Results

Table: Performance Comparison (Average Values, based on thesis Table 5.1)

| Metric | Config I (Baseline) | Config II (Dynamic) | Config III (Dyn.+Mux.) |
|---|---|---|---|
| **CPU Usage (% Avg)** | | | |
| XPLOG++ Agent | 2.95% | **0.25%** | 0.26% |
| XPLOG++ Collector | 21.10% | **5.15%** | 5.17% |
| **Memory Usage (RSS)** | | | |
| XPLOG++ Agent | 792 KiB | **464 KiB** | 470 KiB |
| XPLOG++ Collector | 350 KiB | 350 KiB | 350 KiB |
| **Log File Size (Bytes)** | 5,645,319 | **156,356** | 156,356 |
| **Log Latency (ms, Avg)** | 38 ms | **10 ms** | 11 ms |

# Performance Results

Table: Performance Comparison (Average Values, based on thesis Table 5.1)

| Metric | Config I (Baseline) | Config II (Dynamic) | Config III (Dyn.+Mux.) |
|---|---|---|---|
| **CPU Usage (% Avg)** | | | |
| XPLOG++ Agent | 2.95% | **0.25%** | 0.26% |
| XPLOG++ Collector | 21.10% | **5.15%** | 5.17% |
| **Memory Usage (RSS)** | | | |
| XPLOG++ Agent | 792 KiB | **464 KiB** | 470 KiB |
| XPLOG++ Collector | 350 KiB | 350 KiB | 350 KiB |
| **Log File Size (Bytes)** | 5,645,319 | **156,356** | 156,356 |
| **Log Latency (ms, Avg)** | 38 ms | **10 ms** | 11 ms |

## Key Findings:

- **Dynamic Logging (I vs II):** Massive reduction in CPU, Memory (Agent), Log Size (>97%), and Latency (∼74%). Highly effective.
- **Multiplexing Overhead (II vs III):** Negligible increase in CPU, Memory, and Latency. Very lightweight feature for handling multiple service streams.

# Conclusion

- Presented **XPLOG++**, enhancing XPLOG for 6G CoD.
- **Contribution 1: Dynamic Logging**
  - Provides fine-grained control over logged syscalls.
  - Significantly reduces performance overhead (CPU, Mem, Network, Latency).
- **Contribution 2: Log Multiplexing**
  - Enables efficient handling of **diverse service streams** (apps, metrics, etc.) on shared edge nodes via one Agent.
  - Adds negligible performance cost.
- **Contribution 3: Proposed CoD Architecture**
  - Practical design using Kubernetes, standard monitoring (Prometheus/Grafana), CAMARA APIs.
  - Positions XPLOG++ as a crucial tool for deep diagnostics and root cause analysis.
- **Result:** XPLOG++ is an efficient and flexible observability framework suitable for demanding, multi-service edge environments.

# Future Work

- **Resource Metric Integration:** Embed key resource usage (CPU, Mem) directly into XPLOG++ logs for richer context per service.
- **Predictive Autoscaling:** Utilize historical patterns from XPLOG++ logs (e.g., syscall frequency, I/O patterns per ServiceID) to build models for proactive scaling.
- **Advanced Edge Scheduling:** Develop custom Kubernetes scheduler plugins informed by XPLOG++ insights (e.g., detailed resource needs per ServiceID, inter-service communication patterns) for more optimized edge placements.

# Thank You

Questions?